

```
import numpy as np
import matplotlib.pyplot as plt
import random

from scipy.fft import fft, ifft, fftfreq, fftshift
from scipy.signal import welch

from QAM_mapper import QAM_mapper
from MSK_mapper import MSK_mapper
from plotting_funcs import plot_constellation

PI = np.pi

def occupied_bandwidth_99(x, fs, threshold=0.99):

    N = len(x)
    X = fft(x)

    P = (np.abs(X)/N)**2

    f = fftfreq(N, 1/fs)

    order = np.argsort(np.abs(f))
    f_sorted = np.abs(f)[order]
    P_sorted = P[order]

    df = fs / N

    cum_power = np.cumsum(P_sorted) * df
    total_power = np.sum(P) * df

    idx = np.searchsorted(cum_power, threshold * total_power)
    f_edge = f_sorted[idx]

    return f_edge

## Model parameters

T_signal = 1e-2
dt = 1e-5
df = 1/dt
numb_points = int(T_signal / dt)
t = np.linspace(0, T_signal, num=numb_points, endpoint=False)
Energy_of_impulse = 1
h = 0.5

bit_rate = 1e4
numb_bits = int(T_signal * bit_rate * np.log2(16) * np.log2(64) * np.log2(256))
bit_sequence = [random.randint(0, 1) for _ in range(numb_bits)]

## Create objects
qam4 = QAM_mapper(4, bit_rate, Energy_of_impulse)
qam16 = QAM_mapper(16, bit_rate, Energy_of_impulse)
qam64 = QAM_mapper(64, bit_rate, Energy_of_impulse)
qam256 = QAM_mapper(256, bit_rate, Energy_of_impulse)

msk = MSK_mapper(bit_rate, Energy_of_impulse, h)

## Find baseband signals
baseband_qam4_signal = qam4.get_baseband_signal(bit_sequence)
baseband_qam16_signal = qam16.get_baseband_signal(bit_sequence)
baseband_qam64_signal = qam64.get_baseband_signal(bit_sequence)
baseband_qam256_signal = qam256.get_baseband_signal(bit_sequence)

baseband_qam4_signal_t = baseband_qam4_signal(t)
baseband_qam16_signal_t = baseband_qam16_signal(t)
baseband_qam64_signal_t = baseband_qam64_signal(t)
baseband_qam256_signal_t = baseband_qam256_signal(t)

baseband_msk_signal_t, t_msk, msk_phases = msk.get_modulate_sequence_and_time_and_phase(0, T_signal, numb_points, bit_sequence)

if np.array_equal(t, t_msk): print("ploho")

## Find spectrums
N = np.size(baseband_qam4_signal_t)
freqs = fftfreq(N, dt)

qam4_spec = fft(baseband_qam4_signal_t) / N
qam16_spec = fft(baseband_qam16_signal_t) / N
qam64_spec = fft(baseband_qam64_signal_t) / N
qam256_spec = fft(baseband_qam256_signal_t) / N

msk_spec = fft(baseband_msk_signal_t) / N

## Find teoretic spectrum effectivity

# find bandwidth where 99% power of all spector's power
qam4_bandwidth = 2*occupied_bandwidth_99(baseband_qam4_signal_t, df)
qam16_bandwidth = 2*occupied_bandwidth_99(baseband_qam16_signal_t, df)
qam64_bandwidth = 2*occupied_bandwidth_99(baseband_qam64_signal_t, df)
qam256_bandwidth = 2*occupied_bandwidth_99(baseband_qam256_signal_t, df)

msk_bandwidth = 2*occupied_bandwidth_99(baseband_msk_signal_t, df)

qam4_spec_eff = bit_rate / qam4_bandwidth
qam16_spec_eff = bit_rate / qam16_bandwidth
qam64_spec_eff = bit_rate / qam64_bandwidth
qam256_spec_eff = bit_rate / qam256_bandwidth

msk_spec_eff = bit_rate / msk_bandwidth

qam4_spec_eff_teor = np.log2(4)
qam16_spec_eff_teor = np.log2(16)
qam64_spec_eff_teor = np.log2(64)
qam256_spec_eff_teor = np.log2(256)

msk_spec_eff_teor = 1 / 1 # bit_rate_msk / bandwidth

print("Bandwidth efficiëntivity:")
print("BW_E QAM4=", qam4_spec_eff, " BW_E QAM4 teoreical=", qam4_spec_eff_teor)
print("BW_E QAM16=", qam16_spec_eff, " BW_E QAM16 teoreical=", qam16_spec_eff_teor)
print("BW_E QAM64=", qam64_spec_eff, " BW_E QAM64 teoreical=", qam64_spec_eff_teor)
print("BW_E QAM256=", qam256_spec_eff, " BW_E QAM256 teoreical=", qam256_spec_eff_teor)

print("BW_E MSK=", msk_spec_eff, " BW_E MSK teoreical=", msk_spec_eff_teor)
```

```
## Find PSD params
freq_psd_qam4, psd_qam4 = welch(baseband_qam4_signal_t, df, window='hann', nperseg=1024, scaling='density')
freq_psd_qam16, psd_qam16 = welch(baseband_qam16_signal_t, df, window='hann', nperseg=1024, scaling='density')
freq_psd_qam64, psd_qam64 = welch(baseband_qam64_signal_t, df, window='hann', nperseg=1024, scaling='density')
freq_psd_qam256, psd_qam256 = welch(baseband_qam256_signal_t, df, window='hann', nperseg=1024, scaling='density')

freq_psd_msk, psd_msk = welch(baseband_msk_signal_t, df, window='hann', nperseg=1024, scaling='density')

## Plotting signals's Amp and Phases

# plotting qam
fig1, axs = plt.subplots(2, 1, figsize=(10, 8))

axs[0].plot(t, np.abs(baseband_qam4_signal_t), '-', label='qam 4')
axs[0].plot(t, np.abs(baseband_qam16_signal_t), '-', label='qam 16')
axs[0].plot(t, np.abs(baseband_qam64_signal_t), '-', label='qam 64')
axs[0].plot(t, np.abs(baseband_qam256_signal_t), '-', label='qam 256')
axs[0].set_title('Amplitudes of QAM')
axs[0].set_xlabel('time sec')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].plot(t, np.angle(baseband_qam4_signal_t), '-', label='qam 4')
axs[1].plot(t, np.angle(baseband_qam16_signal_t), '-', label='qam 16')
axs[1].plot(t, np.angle(baseband_qam64_signal_t), '-', label='qam 64')
axs[1].plot(t, np.angle(baseband_qam256_signal_t), '-', label='qam 256')
axs[1].set_title('Phases of QAM')
axs[1].set_xlabel('time sec')
axs[1].set_ylabel('Radians')
axs[1].legend()
axs[1].grid(True)

plt.tight_layout()
plt.show()

# plotting msk
fig2, axs = plt.subplots(2, 1, figsize=(10, 8))

x_vals = np.arange(0, T_signal, msk._T_symb)
axs[1].vlines(x_vals, np.min(msk_phases), np.max(msk_phases), linestyle='dashed')

axs[0].plot(t_msk, np.real(baseband_msk_signal_t), '-', label='msk real')
axs[0].plot(t_msk, np.imag(baseband_msk_signal_t), '-', label='msk imag')
axs[0].set_title('Baseband amplitudes of MSK')
axs[0].set_xlabel('time sec')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].plot(t_msk, msk_phases, '-', label='msk')
axs[1].set_title('Baseband Phases of MSK')
axs[1].set_xlabel('time sec')
axs[1].set_ylabel('Radians')
axs[1].legend()
axs[1].grid(True)

plt.tight_layout()
plt.show()

## Plotting spectrums
# plotting qam
fig3, axs = plt.subplots(4, 1, figsize=(16, 8))

axs[0].axvline(qam4_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[0].axvline(-qam4_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[0].plot(freqs, np.abs(qam4_spec), '--')
axs[0].set_title('Amplitudes of QAM4')
axs[0].set_xlabel('freq Hz')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].axvline(qam16_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[1].axvline(-qam16_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[1].plot(freqs, np.abs(qam16_spec), '--')
axs[1].set_title('Amplitudes of QAM16')
axs[1].set_xlabel('freq Hz')
axs[1].set_ylabel('Amp')
axs[1].legend()
axs[1].grid(True)

axs[2].axvline(qam64_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[2].axvline(-qam64_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[2].plot(freqs, np.abs(qam64_spec), '--')
axs[2].set_title('Amplitudes of QAM64')
axs[2].set_xlabel('freq Hz')
axs[2].set_ylabel('Amp')
axs[2].legend()
axs[2].grid(True)

axs[3].axvline(qam256_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[3].axvline(-qam256_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[3].plot(freqs, np.abs(qam256_spec), '--')
axs[3].set_title('Amplitudes of QAM256')
axs[3].set_xlabel('freq Hz')
axs[3].set_ylabel('Amp')
axs[3].legend()
axs[3].grid(True)

plt.tight_layout()
plt.show()

# plotting msk
fig4, axs = plt.subplots(2, 1, figsize=(10, 8))

axs[0].axvline(msk_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[0].axvline(-msk_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[0].plot(freqs, np.abs(msk_spec), '--')
axs[0].set_title('MSK spectrum\'s amplitude')
axs[0].set_xlabel('freq Hz')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].plot(freqs, np.angle(msk_spec), '--')
axs[1].set_title('MSK spectrum\'s phase')
axs[1].set_xlabel('freq Hz')
axs[1].set_ylabel('Radians')
```

```

    axs[1].grid(True)

plt.tight_layout()
plt.show()

## Plotting PSD
# QAM4
fig5, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam4, psd_qam4, '--', label='QAM 4')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM4 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()

# QAM16
fig6, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam16, psd_qam16, '--', label='QAM 16')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM16 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()

# QAM64
fig7, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam64, psd_qam64, '--', label='QAM 64')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM64 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()

# QAM256
fig8, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam256, psd_qam256, '--', label='QAM 256')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM256 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()
```